

Debacu Revenue Intelligence

Documento base de Fase 1

Tablas, relaciones y políticas

Base de referencia para continuar la implementación técnica de Revenue Intelligence
sin duplicar Stripe, planes, organizaciones, members ni la seguridad actual de Debacu.

Principio rector: Revenue Intelligence se apoya en el core actual de Debacu. No crea un sistema paralelo de suscripción, organizaciones o acceso; añade únicamente las tablas y procesos necesarios para propiedades, configuración operativa, datos diarios, pickup e importación CSV.

1. Objetivo del documento

Este documento fija la base funcional y técnica de la Fase 1 de Revenue Intelligence en Debacu. Su objetivo es que el siguiente trabajo —SQL, políticas RLS, servicios, Edge Functions e importación CSV— siga una misma lógica y no se desvíe del modelo que ya funciona en Debacu Evaluation.

- Mantener intacto el core actual de Debacu: suscripciones, Stripe, organizaciones, miembros, JWT y control por plan.
- Añadir el módulo de Revenue Intelligence de forma progresiva, sin reemplazos bruscos ni duplicidades innecesarias.
- Empezar por la capa mínima útil: propiedades, tipos de habitación, temporadas, eventos, revenue diario, pickup e importaciones.
- Dejar fuera en esta fase todo lo que complique sin aportar valor inmediato: RMS avanzado, forecast complejo, pricing engine o modelo completo de reservas granulares.

2. Principios de diseño

- No duplicar entidades existentes. Revenue usa el modelo actual de Debacu para organización, membresía, permisos y planes.
- Modelo incremental. Se habilita una Fase 1 productiva mientras el resto de Debacu sigue estable y listo para producción con hoteles piloto.
- Mismo patrón de seguridad. Todo debe seguir la filosofía actual: JWT, org_id, membresías y RLS por organización.
- CSV primero, PMS/API después. La ingesta inicial se apoya en importaciones robustas, trazables y repetibles.
- Métricas fiables antes que complejidad. Se prioriza revenue diario y snapshots de pickup antes que un modelo exhaustivo de reservas.

3. Alcance funcional de Fase 1

La Fase 1 debe permitir construir y alimentar las primeras pantallas del módulo Revenue Intelligence:

- Propiedades

- Dashboard
- Día a día
- Pickup avanzado
- Canales & Segmentos
- Importación CSV

Informes podrá apoyarse en esta base, pero no necesita un motor completo adicional en esta fase. La pantalla Mi cuenta no forma parte del módulo de Revenue porque ya existe en Debacu y está conectada a Stripe y al control de planes.

4. Modelo de datos base de Fase 1

Las tablas de Fase 1 se dividen en tres bloques: configuración de propiedades, datos operativos de revenue y pipeline de importación.

Tabla	Bloque	Propósito operativo	Estado esperado en Fase 1
debacu_eval_properties	Configuración	Catálogo de propiedades por organización.	Obligatoria
debacu_eval_property_room_types	Configuración	Tipos de habitación por propiedad.	Obligatoria
debacu_eval_property_seasons	Configuración	Temporadas por propiedad.	Obligatoria
debacu_eval_revenue_events	Configuración	Eventos con impacto en demanda o precio.	Obligatoria
debacu_eval_revenue_daily	Datos	Dato diario consolidado para dashboard, día a día y canales/segmentos.	Obligatoria
debacu_eval_revenue_pickup_snapshots	Datos	Fotos históricas para cálculo de pickup.	Obligatoria
debacu_eval_revenue_imports	Ingesta	Cabecera de importación CSV con estado y trazabilidad.	Obligatoria
debacu_eval_revenue_import_errors	Ingesta	Detalle de errores por fila/campo en importación.	Obligatoria

4.1 Propiedades

debacu_eval_properties guarda la relación entre organización y propiedad operativa. No sustituye al modelo central de organizaciones; lo extiende para Revenue.

- Relación principal: org_id → debacu_eval_organizations(id).
- Campos funcionales mínimos: code, name, city, country, timezone, rooms_total, is_active.
- Campos auxiliares útiles: legal_name, tax_id, property_type, region, currency_code, created_by.

- Regla de negocio: el número de propiedades activas se controlará por plan, pero esa validación se hará en servicio/Edge Function, no duplicando lógica de planes en SQL.

4.2 Tipos de habitación

debacu_eval_property_room_types permite estructurar la oferta de cada propiedad y preparar después lógica de pricing, segmentación y análisis por inventario.

- Relaciones: org_id → organizations; property_id → properties.
- Campos clave: code, name, capacity, rooms_count, base_price, is_active.
- Restricción esperada: unique(property_id, code).
- Uso posterior: filtros de dashboard, revenue diario por tipo de habitación y formularios de configuración.

4.3 Temporadas

debacu_eval_property_seasons modela bloques temporales operativos por propiedad.

- Relaciones: org_id y property_id.
- Campos clave: season_type, start_date, end_date, priority, is_active.
- Regla mínima: end_date >= start_date.
- Uso posterior: vista comercial, reglas futuras de pricing y análisis contextual del revenue.

4.4 Eventos

debacu_eval_revenue_events registra contexto externo o interno con impacto potencial en demanda y precio.

- Relaciones: org_id y property_id.
- Campos clave: name, event_type, impact_level, start_date, end_date, note, is_active.
- Uso posterior: cruce entre ocupación/pickup y eventos relevantes.

4.5 Revenue diario

debacu_eval_revenue_daily es la tabla central del módulo en Fase 1.

- Relaciones: org_id, property_id y room_type_id opcional.
- Grano esperado: property + stay_date + channel + segment + room_type.
- Campos clave: rooms_sold, rooms_available, revenue_rooms, revenue_total, adr, revpar, source.
- Pantallas que alimenta: Dashboard, Día a día y Canales & Segmentos.

4.6 Pickup snapshots

debacu_eval_revenue_pickup_snapshots conserva fotografías históricas del on-the-books para poder calcular pickup real.

- Relaciones: org_id y property_id.
- Campos clave: snapshot_date, stay_date, channel, segment, rooms_sold, revenue_rooms.
- Grano esperado: property + snapshot_date + stay_date + channel + segment.
- Pantalla que alimenta: Pickup avanzado.

4.7 Importaciones

debacu_eval_revenue_imports y debacu_eval_revenue_import_errors dan trazabilidad a la entrada de datos.

- Import cabecera: tipo, estado, ruta, hash, contadores, resumen de errores y metadatos.
- Import errores: fila, campo, código y mensaje.
- Patrón recomendado: validar primero, registrar errores, y solo después hacer commit de los datos buenos.

5. Relaciones y dependencia con el core actual de Debacu

Revenue Intelligence no define su propio sistema de acceso ni su propio catálogo de planes. La dependencia correcta con el core actual es la siguiente:

Entidad base ya existente	Uso desde Revenue
debacu_eval_organizations	Entidad raíz de tenant; todas las tablas de Revenue cuelgan de org_id.
debacu_eval_org_members	Base de autorización y RLS por membresía/rol.
JWT / auth.uid()	Identificación del usuario autenticado.
Sistema actual de suscripción / Stripe	Control de plan, límites y entitlements. Revenue lo consulta; no lo duplica.

La validación de límites —por ejemplo, cuántas propiedades puede crear cada organización según plan— debe ejecutarse en servicios o Edge Functions con service role, leyendo el plan real de Debacu. No debe materializarse un segundo catálogo de planes dentro de Revenue.

6. Política de seguridad y RLS

La seguridad de Fase 1 debe seguir exactamente el patrón del resto de Debacu: aislamiento por organización, control por membresía y escritura restringida por rol.

Capa	Regla base
Frontend	Mostrar/ocultar pantallas según plan y permisos, pero sin confiar solo en la UI.
Ruta	No permitir acceso por URL directa a una opción no habilitada.
Base de datos	RLS habilitado en todas las tablas de Revenue.
Policies de lectura	Un usuario autenticado solo ve registros de organizaciones donde es miembro.
Policies de escritura	Solo OWNER y ADMIN pueden crear/editar/borrar configuración o datos importados.
Edge Functions	Usar service role solo cuando haga falta validar límites, importar CSV o hacer procesos internos.

Helpers recomendados de RLS: una función para comprobar acceso a una organización y otra para comprobar si el usuario es OWNER o ADMIN en esa organización. Todas las policies de Revenue deben colgar de ese patrón.

7. Orden de implementación recomendado

Orden	Pieza	Motivo
-------	-------	--------

1	Cerrar SQL base y RLS	Evita seguir construyendo sobre tablas incompletas o sin seguridad.
2	Pantalla Propiedades	Es la base del módulo y el punto de entrada de configuración.
3	Edge Function de gestión de propiedades	Debe validar org, rol y límite por plan.
4	Importación CSV Revenue Daily	Es la forma más realista de empezar a alimentar datos.
5	Importación CSV Pickup	Permite activar Pickup Avanzado sin depender de PMS integrado.
6	Dashboard	Da valor visual rápido con datos ya consolidados.
7	Día a día	Consulta directa sobre revenue_daily.
8	Canales & Segmentos	Agregación sobre revenue_daily.
9	Pickup avanzado	Comparativa de snapshots.

8. Qué queda fuera de Fase 1

- Duplicar My Account, planes o gestión Stripe dentro de Revenue Intelligence.
- Crear un segundo modelo de organizaciones o miembros.
- Meter un RMS completo con pricing automático, overbooking, optimización avanzada o forecast complejo.
- Forzar desde el inicio una integración PMS universal. La prioridad es CSV robusto y trazable.
- Basar el módulo en reservas granulares si eso retrasa la entrega. En Fase 1 manda el dato útil y consistente.

9. Checklist de cierre de Fase 1 base

Ítem	Criterio de cierre
Tablas	Las ocho tablas de Fase 1 existen y tienen sus columnas clave.
Relaciones	Todas las FK principales contra organizations, properties y room_types están creadas.
RLS	RLS activo en todas las tablas de Revenue.
Policies	Policies SELECT y WRITE creadas y probadas.
Importación	Cabecera y errores de importación operativos.

Frontend	Pantalla Propiedades en funcionamiento.
Datos	Al menos una propiedad con room types, temporadas, eventos y una importación válida de revenue/pickup.

10. Decisión base para continuar

La decisión técnica correcta para Debacu es continuar Revenue Intelligence como módulo nuevo, apoyado en el core existente y alimentado en Fase 1 por propiedades, configuración operativa, revenue diario, pickup e importación CSV. Este documento sirve como referencia para que los siguientes SQL, servicios y Edge Functions mantengan coherencia y no introduzcan duplicidades ni desviaciones arquitectónicas.

Seguimos

```
-- =====
```

```
-- 1) TABLA: debacu_eval_room_prices
```

```
-- =====
```

```
create extension if not exists pgcrypto;
```

```
create table if not exists public.debacu_eval_room_prices (
  id uuid primary key default gen_random_uuid(),
```

```
  property_id uuid not null
  references public.debacu_eval_properties(id)
  on delete cascade,
```

```
  room_type_id uuid not null
  references public.debacu_eval_property_room_types(id)
  on delete cascade,
```

```
  date date not null,
```

```
  price numeric(10,2) not null,
```

min_stay integer not null default 1,
closed boolean not null default false,

created_at timestamptz not null default now(),
updated_at timestamptz not null default now(),

constraint debacu_eval_room_prices_price_chk
check (price >= 0),

constraint debacu_eval_room_prices_min_stay_chk
check (min_stay >= 1)

);

-- =====

-- 2) UNICIDAD

-- Un precio por property + room_type + date

-- =====

create unique index if not exists debacu_eval_room_prices_property_room_type_date_idx
on public.debacu_eval_room_prices (property_id, room_type_id, date);

-- =====

-- 3) ÍNDICES ÚTILES

-- =====

create index if not exists debacu_eval_room_prices_property_date_idx
on public.debacu_eval_room_prices (property_id, date);

create index if not exists debacu_eval_room_prices_room_type_date_idx
on public.debacu_eval_room_prices (room_type_id, date);

create index if not exists debacu_eval_room_prices_property_room_type_idx
on public.debacu_eval_room_prices (property_id, room_type_id);

```
-- =====  
-- 4) VALIDACIÓN DE CONSISTENCIA:  
-- room_type_id debe pertenecer al mismo property_id  
-- =====
```

```
create or replace function public.debacu_eval_room_prices_validate_room_type_property()
```

```
returns trigger
```

```
language plpgsql
```

```
as $$
```

```
declare
```

```
    v_exists boolean;
```

```
begin
```

```
    select exists (
```

```
        select 1
```

```
        from public.debacu_eval_property_room_types rt
```

```
        where rt.id = new.room_type_id
```

```
            and rt.property_id = new.property_id
```

```
    )
```

```
    into v_exists;
```

```
    if not v_exists then
```

```
        raise exception
```

```
            'room_type_id % does not belong to property_id %',
```

```
            new.room_type_id, new.property_id;
```

```
    end if;
```

```
    return new;
```

```
end;
```

```
$$;
```

```
drop trigger if exists trg_debacu_eval_room_prices_validate_room_type_property
```

```
on public.debacu_eval_room_prices;
```

```
create trigger trg_debacu_eval_room_prices_validate_room_type_property
```



```

before insert or update on public.debacu_eval_room_prices
for each row
execute function public.debacu_eval_room_prices_validate_room_type_property();

-- =====
-- 5) updated_at automático
-- =====

create or replace function public.set_debacu_eval_room_prices_updated_at()
returns trigger
language plpgsql
as $$
begin
    new.updated_at := now();
    return new;
end;
$$;

drop trigger if exists trg_debacu_eval_room_prices_updated_at
on public.debacu_eval_room_prices;

create trigger trg_debacu_eval_room_prices_updated_at
before update on public.debacu_eval_room_prices
for each row
execute function public.set_debacu_eval_room_prices_updated_at();

-- =====
-- 6) RLS
-- =====

alter table public.debacu_eval_room_prices enable row level security;

-- SELECT
drop policy if exists debacu_eval_room_prices_select on public.debacu_eval_room_prices;

```

```
create policy debacu_eval_room_prices_select
on public.debacu_eval_room_prices
for select
to authenticated
using (
  exists (
    select 1
    from public.debacu_eval_properties p
    where p.id = debacu_eval_room_prices.property_id
    and public.debacu_eval_has_org_access(p.org_id)
  )
);
```

-- INSERT

```
drop policy if exists debacu_eval_room_prices_insert on public.debacu_eval_room_prices;
create policy debacu_eval_room_prices_insert
on public.debacu_eval_room_prices
for insert
to authenticated
with check (
  exists (
    select 1
    from public.debacu_eval_properties p
    where p.id = debacu_eval_room_prices.property_id
    and public.debacu_eval_has_org_access(p.org_id)
  )
);
```

-- UPDATE

```
drop policy if exists debacu_eval_room_prices_update on public.debacu_eval_room_prices;
create policy debacu_eval_room_prices_update
on public.debacu_eval_room_prices
for update
to authenticated
```

```

using (
  exists (
    select 1
    from public.debacu_eval_properties p
    where p.id = debacu_eval_room_prices.property_id
    and public.debacu_eval_has_org_access(p.org_id)
  )
)
with check (
  exists (
    select 1
    from public.debacu_eval_properties p
    where p.id = debacu_eval_room_prices.property_id
    and public.debacu_eval_has_org_access(p.org_id)
  )
);

```

-- DELETE

drop policy if exists debacu_eval_room_prices_delete on public.debacu_eval_room_prices;

create policy debacu_eval_room_prices_delete

on public.debacu_eval_room_prices

for delete

to authenticated

```

using (
  exists (
    select 1
    from public.debacu_eval_properties p
    where p.id = debacu_eval_room_prices.property_id
    and public.debacu_eval_has_org_access(p.org_id)
  )
);

```

-- =====

-- 7) COMENTARIOS

-- =====

comment on table public.debacu_eval_room_prices is

'Precios diarios por tipo de habitación y propiedad para Revenue Intelligence / mini RMS';

comment on column public.debacu_eval_room_prices.price is

'Precio diario para la fecha y tipo de habitación';

comment on column public.debacu_eval_room_prices.min_stay is

'Estancia mínima requerida para esa fecha';

comment on column public.debacu_eval_room_prices.closed is

'Marca si el tipo de habitación está cerrado para venta en esa fecha';

Debacu Revenue Intelligence

Documento base de Fase 2

He leído el documento base de Fase 1 y encaja con lo que has montado: Revenue Intelligence debe apoyarse en el core actual de Debacu, usar propiedades, tipos, temporadas, eventos, revenue diario, pickup e importación, sin duplicar suscripciones, miembros ni seguridad. Además, el propio documento deja claro que Fase 1 no era un RMS completo, sino una base útil y progresiva.

Documento_base_Fase1_Revenue_In...

Viendo también las pantallas mock que has pasado, la conclusión práctica es esta:

la siguiente fase correcta no es un bloque aislado, sino una Fase 2 centrada en “semántica real + datos reales”.

Si me obligas a elegir entre tus dos rutas:

primero Ruta A: Eventos / temporadas reales
y después Ruta B: bloqueos reales por reserva / mantenimiento.

La razón no es teórica, es práctica:

- **Ruta A tiene menos dependencia externa.** La puedes cerrar con tus tablas y Edge Functions sin necesitar todavía un modelo fino de reservas.
- **Da valor visual y comercial inmediato.** Tu calendario de precios deja de ser “bonito” y pasa a tener lógica de negocio visible.
- **Alimenta varias pantallas a la vez:** PriceCalendar, Eventos, Dashboard, Día x Día, comparativas futuras.
- **Prepara pricing rules y forecasting.** Sin temporadas y eventos reales, cualquier recomendación futura de precio va coja.
- **Ruta B es más delicada** porque exige una fuente fiable de reservas, estados, cierres y prioridad de bloqueo. Si lo haces antes de tener bien la semántica, te vas a meter en demasiadas excepciones.

Dicho claro:

si haces primero bloqueos por reserva sin tener bien modelados temporadas, eventos y contexto diario, conviertes el calendario en una herramienta operativa parcial.

Si haces primero temporadas/eventos reales, conviertes el calendario en una base seria sobre la que luego sí tiene sentido bloquear, restringir y automatizar.

Mi propuesta realista de Fase 2

Fase 2A — Semántica real del calendario y base de analítica

Objetivo: que el calendario y las pantallas mock de Revenue dejen de depender de simulación.

Bloque 1:

Eventos y temporadas reales

- CRUD real de temporadas por propiedad
- CRUD real de eventos por propiedad
- color persistido en BD
- prioridad / impacto real
- reflejo visual en PriceCalendar
- leyenda superior alimentada desde BD, no hardcoded

Bloque 2:

Revenue daily y pickup snapshots reales

- empezar a poblar debacu_eval_revenue_daily
- empezar a poblar debacu_eval_revenue_pickup_snapshots
- usar importación CSV primero, como dice el documento base
- conectar las pantallas mock:
 - Día x Día
 - Mensual
 - Canales & Segmentos
 - Pickup Avanzado

Bloque 3:

Entitlements y acceso

- respetar siempre property/org access
- mismas reglas de Edge Functions + RLS ya definidas en Debacu
- sin meter lógica sensible en frontend

Por qué esta Fase 2A es la mejor siguiente

Valor producto

Muy alto. Puedes enseñar algo vendible enseguida:

- calendario con semántica real
- pantalla de eventos útil
- métricas diarias reales

- pickup real
- canales/segmentos reales

Complejidad

Media. No trivial, pero controlable.

No exige todavía modelar reservas unitarias ni estados de bloqueo complejos.

Dependencia de datos

Moderada.

Con CSV puedes avanzar ya. No dependes todavía de PMS API universal.

Impacto técnico

Bueno.

Construyes la base correcta para el resto en vez de improvisar lógica operativa encima de mocks.

Qué dejaría para Fase 2B inmediatamente después

Bloqueos reales por reserva / mantenimiento

Esto sería el siguiente paso, no el primero.

Aquí ya sí metería:

- blocked_reason
- blocked_source
- reservation_lock
- prioridad de estados por día
- impedir edición si el día está vendido o bloqueado
- diferenciar:
 - manual_close
 - maintenance
 - reservation_hold / sold
- conflictos en edición masiva

Pero hacerlo antes de 2A me parece precipitado.

Diseño recomendado de Fase 2A

1) Tablas a consolidar ya

El documento base ya contempla estas piezas como obligatorias o núcleo de Fase 1: propiedades, room types, seasons, events, revenue daily y pickup snapshots.

Documento_base_Fase1_Revenue_In...

Yo cerraría primero estas dos si aún no están completas a nivel de uso real:

debacu_eval_property_seasons

Campos recomendados:

```
create table if not exists public.debacu_eval_property_seasons (  
  id uuid primary key default gen_random_uuid(),  
  org_id uuid not null references public.debacu_eval_organizations(id) on delete  
cascade,  
  property_id uuid not null references public.debacu_eval_properties(id) on  
delete cascade,  
  name text not null,  
  season_type text not null,  
  start_date date not null,  
  end_date date not null,  
  color text not null default '#3B82F6',  
  priority integer not null default 100,  
  impact_level text null,  
  note text null,  
  is_active boolean not null default true,  
  created_at timestamptz not null default now(),  
  updated_at timestamptz not null default now(),  
  constraint debacu_eval_property_seasons_dates_chk check (end_date >=  
start_date)  
);  
create index if not exists debacu_eval_property_seasons_property_dates_idx  
on public.debacu_eval_property_seasons(property_id, start_date, end_date);  
create unique index if not exists  
debacu_eval_property_seasons_property_name_dates_uniq  
on public.debacu_eval_property_seasons(property_id, name, start_date,  
end_date);
```

debacu_eval_revenue_events

```
create table if not exists public.debacu_eval_revenue_events (  
  id uuid primary key default gen_random_uuid(),  
  org_id uuid not null references public.debacu_eval_organizations(id) on delete  
cascade,  
  property_id uuid not null references public.debacu_eval_properties(id) on  
delete cascade,  
  name text not null,  
  event_type text not null,
```



```

start_date date not null,
end_date date not null,
color text not null default '#10B981',
impact_level text not null default 'MEDIUM',
priority integer not null default 200,
note text null,
is_active boolean not null default true,
created_at timestamptz not null default now(),
updated_at timestamptz not null default now(),
constraint debacu_eval_revenue_events_dates_chk check (end_date >= start_date),
constraint debacu_eval_revenue_events_impact_chk check (impact_level in
('LOW', 'MEDIUM', 'HIGH'))
);
create index if not exists debacu_eval_revenue_events_property_dates_idx
on public.debacu_eval_revenue_events(property_id, start_date, end_date);

```

2) Vista útil para pintar el calendario

Para no meter lógica absurda en frontend, crearía una vista que resuelva qué etiqueta/color aplica a cada fecha:

```

create or replace view public.debacu_eval_property_calendar_context_v as
with season_days as (
  select
    s.org_id,
    s.property_id,
    gs::date as calendar_date,
    'SEASON'::text as source_type,
    s.id as source_id,
    s.name,
    s.season_type as item_type,
    s.color,
    s.priority,
    s.impact_level
  from public.debacu_eval_property_seasons s,
       generate_series(s.start_date, s.end_date, interval '1 day') gs
  where s.is_active = true
),
event_days as (
  select
    e.org_id,
    e.property_id,
    gs::date as calendar_date,
    'EVENT'::text as source_type,
    e.id as source_id,
    e.name,
    e.event_type as item_type,
    e.color,
    e.priority,
    e.impact_level
  from public.debacu_eval_revenue_events e,
       generate_series(e.start_date, e.end_date, interval '1 day') gs
  where e.is_active = true

```

```

),
unioned as (
  select * from season_days
  union all
  select * from event_days
),
ranked as (
  select *,
    row_number() over (
      partition by property_id, calendar_date
      order by priority asc, source_type asc, name asc
    ) as rn
  from unioned
)
select
  org_id,
  property_id,
  calendar_date,
  source_type,
  source_id,
  name,
  item_type,
  color,
  priority,
  impact_level
from ranked
where rn = 1;

```

Eso te da una sola “capa principal” por día para pintar el puntito/color en el calendario.

3) Edge Functions que haría ahora

debacu_eval_revenue_seasons_manage

Operaciones:

- LIST
- UPSERT
- DELETE
- GET_BY_PROPERTY_RANGE

debacu_eval_revenue_events_manage

Operaciones:

- LIST
- UPSERT
- DELETE
- GET_BY_PROPERTY_RANGE

debacu_eval_revenue_dashboard_get

Que lea de:

- debacu_eval_revenue_daily
- debacu_eval_revenue_pickup_snapshots
- debacu_eval_property_calendar_context_v

debacu_eval_revenue_daily_import

CSV → valida → guarda en debacu_eval_revenue_imports y ..._errors → commit a debacu_eval_revenue_daily

debacu_eval_revenue_pickup_import

CSV → mismo patrón → commit a debacu_eval_revenue_pickup_snapshots

Frontend que tocaría ya

Páginas

Ya existente

- PriceCalendarPage.tsx

Nuevas / consolidar mocks

- RevenueEventsPage.tsx
- RevenueDayByDayPage.tsx
- RevenueMonthlyPage.tsx
- RevenueChannelsSegmentsPage.tsx
- RevenuePickupPage.tsx

Servicios

- revenueSeasons.service.ts
- revenueEvents.service.ts
- revenueDashboard.service.ts
- revenueDaily.service.ts
- revenuePickup.service.ts

En PriceCalendarPage.tsx

Añadir:

- carga de contexto por rango visible
 - merge de:
 - precio/min stay/cerrado
 - color/etiqueta de temporada o evento
 - leyenda real arriba
 - tooltip real con:
 - nombre evento/temporada
 - tipo
 - prioridad
 - impacto
-

Cómo usar tus pantallas mock sin rehacer nada

Tus mocks ya marcan el camino. Yo no los trataría como “diseño separado”, sino como backlog de integración de datos reales:

Pantalla Eventos

Es la primera que conectaría a BD real.

Canales & Segmentos

Depende de `debacu_eval_revenue_daily`.

Pickup Avanzado

Depende de `debacu_eval_revenue_pickup_snapshots`.

Mensual

Se puede alimentar desde agregación sobre `revenue_daily`.

Día x Día

Igual, con agregación diaria por propiedad.

O sea: **las pantallas ya te están diciendo qué tablas hay que poblar primero.**

Orden exacto que te recomiendo

Paso 1

Cerrar definitivamente:

- seasons
- events
- calendar context view

Paso 2

Conectar PriceCalendarPage a:

- room_prices
- calendar_context_v

Paso 3

Montar import de revenue_daily

Paso 4

Conectar:

- Día x Día
- Mensual
- Canales & Segmentos

Paso 5

Montar import de pickup_snapshots

Paso 6

Conectar Pickup Avanzado

Paso 7

Solo entonces:

bloqueos reales por reserva / mantenimiento

Decisión final

Mi recomendación firme es esta:

Haz ahora Fase 2A = Eventos/temporadas reales + revenue_daily + pickup real.
Deja Fase 2B = bloqueos por reserva/mantenimiento justo después.

Porque:

- te da más valor visible por menos riesgo,
- encaja mejor con el documento base,
- aprovecha tus pantallas mock,
- y evita meter lógica operativa dura sobre una base todavía demasiado simulada.

Documento_base_Fase1_Revenue_In...

Si quieres, en el siguiente mensaje te bajo esto a un bloque ya ejecutable con:

SQL completo + Edge Functions + servicios TS + estructura de páginas para arrancar Fase 2A.

Ahora mismo te falta una pieza clave del modelo: **la regla de pricing por temporada/evento.**

Con lo que tienes hoy:

- **Tipos de habitación** = precio base estructural
- **Temporadas / eventos** = contexto de demanda
- **Calendario de precios** = precio final diario editable

Pero todavía no has definido **cómo pasar del precio base al precio sugerido o calculado.**

Dónde debe ir esa lógica

No la pondría en `room_types`, porque ahí solo debe vivir el **precio base estable** de cada tipo.

Tampoco la metería “a mano” en cada día del calendario, porque eso obliga a editar cientos de celdas y no escala.

La pondría en una **capa intermedia de reglas de ajuste**, algo así:

nueva tabla

`debacu_eval_revenue_rules`

o más claro:

`debacu_eval_property_price_rules`

Qué guardaría ahí

Por propiedad, y opcionalmente por tipo de habitación:

- id
- org_id
- property_id
- room_type_id nullable
- applies_to = SEASON | EVENT | BASE
- reference_id = id de temporada o evento
- adjustment_type = PERCENT | FIXED
- adjustment_value = 50 por ejemplo
- operation = INCREASE | DECREASE | SET
- priority
- is_active

Ejemplo real

Habitación doble

precio base = 100 €

Temporada Alta Verano

regla = INCREASE + PERCENT + 50

Resultado sugerido:

150 €

Congreso médico

regla = INCREASE + PERCENT + 30

Si coinciden ambas cosas, decides política:

opción simple

gana la regla de mayor prioridad

opción más flexible

se acumulan hasta un límite

Para empezar, yo haría **gana la de mayor prioridad**. Mucho más controlable.

Cómo lo haría en Debacu

1. Precio base

Se sigue definiendo en **Tipos de habitación**.

Eso ya lo tienes bien.

2. Temporadas y eventos

Siguen siendo contexto puro, con color, prioridad, impacto.

Eso también está bien.

3. Reglas de pricing

Nueva pantalla o bloque dentro de Eventos/Temporadas:

- Temporada Baja → -20%
- Temporada Media → +0%
- Temporada Alta → +50%
- Navidad / Pico → +80%
- Congreso médico → +25%
- Feria → +15%

4. Calendario de precios

El calendario muestra:

- precio actual diario
 - precio sugerido por reglas
 - opción de aplicar automáticamente rango o dejar manual
-

La decisión importante

Tienes dos caminos.

Camino correcto para producto serio

El calendario no recalcula solo al vuelo, sino que hay un proceso:

- precio base
- reglas de temporada/evento
- se genera **precio sugerido**

- el usuario decide **aplicarlo** al calendario

Ventaja:

no se cambian precios sin control.

Camino rápido pero peligroso

Cada vez que veas una temporada, el calendario recalcula automáticamente.

No te lo recomiendo porque:

- pierdes control
 - mezclarás sugerencia con precio real grabado
 - luego será difícil explicar por qué cambió algo
-

Mi recomendación clara

Haz esto:

En tipos de habitación

mantén solo:

- nombre
- capacidad
- habitaciones
- **base_price**

Crea una nueva pantalla:

Reglas de pricing

O más práctico aún, primero como bloque dentro de:

Eventos y temporadas

Por cada temporada/evento:

- tipo de ajuste
- valor
- si afecta a todas las habitaciones o solo a algunas

Ejemplo:

Temporada Alta

- ajuste: +50%
- aplica a: todas

Temporada Baja

- ajuste: -20%
- aplica a: todas

Congreso médico

- ajuste: +30%
- aplica a: suites y dobles

En calendario de precios

Añade un botón tipo:

- **Aplicar reglas al rango**
 - **Aplicar sugerencia**
 - **Previsualizar cambios**
-

En resumen

La subida de julio y agosto **no debe definirse en tipos de habitación**.
Debe definirse en una capa de **reglas comerciales por temporada/evento**.

Modelo correcto

- **Tipos de habitación** → precio base
 - **Temporadas/Eventos** → contexto
 - **Reglas de pricing** → fórmula de ajuste
 - **Calendario de precios** → resultado final diario
-

Lo más sensato que haría ahora

Siguiente fase:

añadir a temporadas/eventos estos campos:

- `adjustment_type = PERCENT | FIXED`
- `adjustment_value`
- `operation = INCREASE | DECREASE | SET`

Así no necesitas otra tabla todavía.

ejemplo

Temporada Alta:

- operation: INCREASE
- adjustment_type: PERCENT
- adjustment_value: 50

Temporada Baja:

- operation: DECREASE
- adjustment_type: PERCENT
- adjustment_value: 20

Para esta fase, **la regla comercial debe vivir en Eventos y Temporadas**. No en tipos de habitación ni en cada celda del calendario.

Pero hay una diferencia importante que conviene dejar clara desde ya:

Hay 3 capas distintas

1. Precio base estructural

Va en **Tipos de habitación**.

Ejemplo:

- Doble = 90 €
- Suite = 140 €

Eso es el precio base “normal” del tipo.

2. Regla comercial planificada

Va en **Temporadas / Eventos**.

Ejemplo:

- Temporada Alta = +50%
- Semana Santa = +35%
- Navidad = +80%
- Congreso Médico = +20%
- Temporada Baja = -15%

Esto define el **precio objetivo o sugerido** para esos días.

3. Precio real publicado / operativo por día

Va en **Calendario de precios**.

Ese es el precio que realmente ofreces ese día.

Lo importante: no mezclar “regla” con “venta real”

Tú has dado un ejemplo muy bueno:

- tienes 5 habitaciones del mismo tipo
- vendes 2 a 95
- luego subes a 120 y vendes 1
- luego bajas a 85 y vendes las últimas

Eso significa que **un mismo día y mismo tipo de habitación puede tener varias ventas a distintos precios.**

Entonces:

el calendario no puede representar “el precio vendido”

El calendario representa:

- el **precio vigente configurado**
- o el **precio objetivo sugerido**
- o ambos

Pero no el histórico real de venta por reserva.

Eso va en otra tabla.

Cómo se debe modelar de verdad

A) Tabla de reglas de pricing

Mi recomendación: añadir campos a **temporadas** y **eventos** en esta fase.

en debacu_eval_property_seasons

añadir:

- pricing_operation → INCREASE | DECREASE | SET
- pricing_adjustment_type → PERCENT | FIXED
- pricing_adjustment_value → numeric

en debacu_eval_revenue_events

igual:

- pricing_operation

- pricing_adjustment_type
- pricing_adjustment_value

Ejemplo

Temporada Alta:

- operation = INCREASE
- adjustment_type = PERCENT
- adjustment_value = 50

Semana Santa:

- operation = INCREASE
- adjustment_type = PERCENT
- adjustment_value = 35

Navidad:

- operation = SET
- adjustment_type = FIXED
- adjustment_value = 220

Esto te permite decir:

- subir un %
- bajar un %
- fijar un precio exacto

Y esto sí encaja con lo que tú quieres.

B) Calendario de precios = precio operativo diario

La tabla del calendario debe guardar el **precio diario vigente por tipo de habitación y fecha**.

Eso ya lo tienes bastante encaminado.

Ejemplo:

- 2026-08-10
- tipo habitación doble
- precio = 135
- min_stay = 2
- closed = false

Ese precio puede venir de:

- base

- base + regla de temporada
 - base + regla de evento
 - edición manual posterior
-

C) Histórico de ventas reales

Aquí necesitas otra tabla distinta. Sí o sí.

Porque si subes un CSV del PMS, lo importante no es solo “hubo reserva”, sino:

- qué fecha se vendió
- qué habitación / tipo
- qué precio se vendió
- qué canal
- qué reserva
- cuántas unidades se vendieron
- cuándo se vendió respecto a la llegada

Nueva tabla recomendada

Algo tipo:

`debacu_eval_revenue_bookings`

o más fino:

`debacu_eval_revenue_booking_lines`

campos mínimos

- `id`
- `org_id`
- `property_id`
- `reservation_id`
- `room_type_id`
- `room_id` nullable
- `channel`
- `rate_plan` nullable
- `stay_date`
- `checkin_date`
- `checkout_date`
- `booking_date`

- units_sold
- price_sold
- currency
- status
- source_file_id o import_batch_id

Con eso ya puedes analizar:

- cuántas habitaciones vendiste
 - a qué precio medio
 - por canal
 - con qué antelación
 - cuánto cambió el precio antes de venderse
-

Lo que preguntas de “cómo se controla si vendo unas a 95, otras a 120 y otras a 85”

La respuesta es:

No se controla en el calendario

Se controla en el **histórico de ventas / booking lines**.

El calendario solo sabe:

- qué precio está configurado ese día

La tabla de ventas sabe:

- qué precio realmente se vendió cada unidad

Y luego sacas métricas:

ejemplo para un día

Tipo doble, 5 habitaciones vendidas:

- 2 a 95
- 1 a 120
- 2 a 85

Entonces puedes calcular:

- unidades vendidas = 5
- ADR real del tipo ese día = $(95+95+120+85+85)/5$
- precio medio real = 96

- rango vendido = 85–120
- desviación respecto al precio objetivo

Eso es lo correcto.

Entonces, ¿qué hace el sistema cuando cambias precios?

El sistema debe guardar dos cosas distintas

1. Estado operativo actual

El precio del calendario para ese día.

2. Histórico de cambios de precio

Esto también deberías guardarlo más adelante.

Tabla recomendada:

`debacu_eval_revenue_price_changes`

campos

- `id`
- `org_id`
- `property_id`
- `room_type_id`
- `calendar_date`
- `old_price`
- `new_price`
- `changed_at`
- `changed_by`
- `reason` (SEASON_RULE, EVENT_RULE, MANUAL, DEMAND_ADJUSTMENT, etc.)

Así puedes saber:

- a qué precio estaba a las 10:00
- cuándo lo subiste
- cuándo lo bajaste
- si la venta ocurrió antes o después del cambio

Sin eso, luego no puedes analizar bien.

Arquitectura realista para Debacu sin PMS API

Como vais a ir por CSV, yo haría esto:

Fase 1

Temporadas / eventos con reglas de pricing

Añadir campos:

- pricing_operation
- pricing_adjustment_type
- pricing_adjustment_value

Fase 2

PriceCalendar genera precio por defecto

Cuando cargas el calendario:

- si no hay precio diario grabado → calcula desde base + regla activa
- si ya hay precio diario → muestra el grabado

Eso te da una operativa muy útil.

Fase 3

Importación CSV de reservas

Crear tabla de reservas vendidas / líneas vendidas

Fase 4

Histórico de cambios de precio

Guardar cada modificación manual o automática

Fase 5

Analítica revenue real

Cruzas:

- precio objetivo
- precio publicado
- precio vendido
- canal

- pickup
- ocupación

Y ahí empieza la parte seria del RMS.

Lo que debe pasar en tu calendario

Tu idea aquí también es correcta:

Si una fecha ya está vendida

No debe dejar modificar esa unidad vendida.

Pero cuidado: aquí hay una trampa.

Si vendiste 2 de 5 habitaciones del tipo

No significa que bloques toda la celda del tipo.

Porque te quedan 3 por vender.

Entonces tienes dos opciones:

opción simple

Si hay cualquier venta ese día para ese tipo, bloqueas la celda completa.

Ventaja:

- muy fácil

Problema:

- no es realista

opción correcta

La celda sigue editable mientras queden unidades disponibles.

Entonces necesitas saber:

- inventario total del tipo
- unidades vendidas ese día
- unidades disponibles = total - vendidas

Y el precio sigue siendo editable para las que quedan disponibles.

Eso es lo que hace sentido.

Mi recomendación clara

Ahora mismo define esto así

En Tipos de habitación

- `base_price`

En Temporadas/Eventos

Añadir:

- `pricing_operation`
- `pricing_adjustment_type`
- `pricing_adjustment_value`

En Calendario

Mostrar:

- precio diario operativo
- que por defecto salga calculado por base + regla
- permitir cambio manual si no está cerrado
- no tocar reservas ya vendidas

Nueva tabla futura imprescindible

- `debacu_eval_revenue_booking_lines`

Otra tabla muy recomendable

- `debacu_eval_revenue_price_changes`
-

Respuesta directa a tu duda

Sí:

- **las subidas por julio/agosto, Navidad, Semana Santa, etc. deben definirse en Eventos y Temporadas**
- **el calendario debe reflejar automáticamente ese precio objetivo por defecto**
- **las ventas reales no se guardan en el calendario, sino en una tabla separada de líneas vendidas**
- **si cambias varias veces el precio en el día, eso solo se podrá analizar bien si guardas histórico de cambios**

Conclusión práctica

Tu planteamiento es bueno, pero necesitas separar:

- **regla de precio**
- **precio diario vigente**
- **precio realmente vendido**

Si no separas esas tres cosas, luego el análisis revenue se vuelve confuso y poco fiable.

El siguiente paso sensato es que te prepare el diseño exacto de esta fase con:

- SQL para ampliar `seasons` y `events`
- cómo calcular el precio por defecto en `PriceCalendar`
- estructura de tabla `booking_lines`
- y reglas para no bloquear toda la celda cuando aún quedan unidades disponibles.

Objetivo de esta fase

Que el sistema haga esto:

- **Tipos de habitación** → guardan el **precio base**
- **Temporadas / eventos** → guardan la **regla comercial**
- **Calendario de precios** → muestra el **precio operativo diario**
- **Reservas importadas por CSV** → guardan el **precio realmente vendido**
- **Cambios de precio** → guardan el histórico de subidas y bajadas

Eso te deja una base seria.

1) Ampliar temporadas y eventos con regla de pricing

Ahora mismo temporadas y eventos ya sirven para contexto visual.

Lo siguiente es que también definan **cómo afectan al precio**.

SQL recomendado

`debacu_eval_property_seasons`

```
alter table public.debacu_eval_property_seasons
add column if not exists pricing_operation text
check (pricing_operation in ('INCREASE', 'DECREASE', 'SET'));
alter table public.debacu_eval_property_seasons
```

```
add column if not exists pricing_adjustment_type text
check (pricing_adjustment_type in ('PERCENT', 'FIXED'));
alter table public.debacu_eval_property_seasons
add column if not exists pricing_adjustment_value numeric(10,2);
comment on column public.debacu_eval_property_seasons.pricing_operation is
'Cómo afecta esta temporada al precio base: INCREASE, DECREASE o SET';
comment on column public.debacu_eval_property_seasons.pricing_adjustment_type is
'Tipo de ajuste: PERCENT (%) o FIXED (importe fijo)';
comment on column public.debacu_eval_property_seasons.pricing_adjustment_value is
'Valor del ajuste. Ej: 50 para +50%, o 120 para SET FIXED 120€';
```

debacu_eval_revenue_events

```
alter table public.debacu_eval_revenue_events
add column if not exists pricing_operation text
check (pricing_operation in ('INCREASE', 'DECREASE', 'SET'));
alter table public.debacu_eval_revenue_events
add column if not exists pricing_adjustment_type text
check (pricing_adjustment_type in ('PERCENT', 'FIXED'));
alter table public.debacu_eval_revenue_events
add column if not exists pricing_adjustment_value numeric(10,2);
comment on column public.debacu_eval_revenue_events.pricing_operation is
'Cómo afecta este evento al precio base: INCREASE, DECREASE o SET';
comment on column public.debacu_eval_revenue_events.pricing_adjustment_type is
'Tipo de ajuste: PERCENT (%) o FIXED (importe fijo)';
comment on column public.debacu_eval_revenue_events.pricing_adjustment_value is
'Valor del ajuste. Ej: 35 para +35%, o 180 para SET FIXED 180€';
```

2) Regla funcional de cálculo

Fórmula simple y válida para esta fase

Partimos de:

- base_price del tipo de habitación

y aplicamos la regla activa del día.

Casos

INCREASE + PERCENT

Base 100, valor 50

→ 150

DECREASE + PERCENT

Base 100, valor 20

→ 80

INCREASE + FIXED

Base 100, valor 25
→ 125

DECREASE + FIXED

Base 100, valor 15
→ 85

SET + FIXED

Base 100, valor 180
→ 180

SET + PERCENT

No tiene mucho sentido.
Yo lo **prohibiría funcionalmente** o lo ignoraría.

3) Qué pasa si coinciden temporada y evento

No conviene sumar reglas todavía. Se complica y puede generar barbaridades.

Regla recomendada para empezar

Gana la regla del contexto con menor priority

igual que ya haces visualmente en la vista de contexto.

O sea:

- si coincide Semana Santa con Temporada Alta
- y Semana Santa tiene prioridad 10
- y Temporada Alta prioridad 100

gana **Semana Santa**

Eso es simple, previsible y comercialmente entendible.

4) Qué debe hacer el calendario de precios

Aquí está la clave.

Comportamiento recomendado

Si NO existe precio grabado en `debacu_eval_room_prices`

el calendario muestra un **precio calculado por defecto**:

- `base_price`
 - regla de temporada/evento si aplica

Si SÍ existe precio grabado

el calendario muestra **ese precio grabado**, porque ya es precio operativo real.

Esto te permite:

- no tener que precargar toda la tabla para todos los días
 - ver precios “inteligentes” desde el principio
 - mantener edición manual cuando quieras
-

5) Diferencia entre precio calculado y precio grabado

Esto hay que mantenerlo claro en código.

Precio calculado

No está persistido todavía. Sale de:

- `base_price`
- regla activa del día

Precio grabado

Está en `debacu_eval_room_prices.price`

Ese ya manda.

Lógica recomendada

```
effectivePrice =  
  roomPriceRow?.price ??  
  calculatedPriceFromBaseAndRules
```

Y para el UI puedes incluso marcar:

- normal → grabado
- sugerido → calculado

Pero para esta fase, con mostrar el efectivo vale.

6) Nueva tabla para ventas reales por CSV

Esto es imprescindible si luego quieres analizar revenue de verdad.

Tabla recomendada

debacu_eval_revenue_booking_lines

```
create table if not exists public.debacu_eval_revenue_booking_lines (  
  id uuid primary key default gen_random_uuid(),  
  org_id uuid not null references public.debacu_eval_organizations(id) on delete  
cascade,  
  property_id uuid not null references public.debacu_eval_properties(id) on  
delete cascade,  
  room_type_id uuid null references public.debacu_eval_property_room_types(id) on  
delete set null,  
  reservation_id text not null,  
  external_reservation_id text null,  
  channel text null,  
  segment text null,  
  rate_plan text null,  
  booking_date date null,  
  checkin_date date not null,  
  checkout_date date not null,  
  stay_date date not null,  
  units_sold integer not null default 1,  
  price_sold numeric(10,2) not null default 0,  
  revenue_rooms numeric(10,2) not null default 0,  
  status text null,  
  currency_code text null default 'EUR',  
  import_batch_id uuid null,  
  source text not null default 'CSV',  
  created_at timestamptz not null default now(),  
  updated_at timestamptz not null default now(),  
  constraint debacu_eval_revenue_booking_lines_units_chk check (units_sold >= 1),  
  constraint debacu_eval_revenue_booking_lines_price_chk check (price_sold >= 0),  
  constraint debacu_eval_revenue_booking_lines_revenue_chk check (revenue_rooms  
>= 0)  
);  
create index if not exists  
debacu_eval_revenue_booking_lines_property_stay_date_idx  
on public.debacu_eval_revenue_booking_lines(property_id, stay_date);  
create index if not exists  
debacu_eval_revenue_booking_lines_property_room_type_stay_date_idx  
on public.debacu_eval_revenue_booking_lines(property_id, room_type_id,  
stay_date);  
create index if not exists debacu_eval_revenue_booking_lines_reservation_idx  
on public.debacu_eval_revenue_booking_lines(reservation_id);
```

7) Cómo modelar las ventas del ejemplo que diste

Ejemplo:

- tipo doble
- 5 habitaciones
- 2 se venden a 95
- 1 se vende a 120
- 2 se venden a 85

Eso no va en una sola fila.

Van varias líneas, por ejemplo:

reservation_id	stay_date	room_type	units_sold	price_sold	revenue_rooms
R1	2026-08-10	DOBLE	1	95	95
R2	2026-08-10	DOBLE	1	95	95
R3	2026-08-10	DOBLE	1	120	120
R4	2026-08-10	DOBLE	1	85	85
R5	2026-08-10	DOBLE	1	85	85

Luego puedes calcular:

- unidades vendidas = 5
- ADR real = 96
- min vendido = 85
- max vendido = 120

Eso sí es real.

8) Nueva tabla para histórico de cambios de precio

Muy recomendable. No imprescindible para hoy, pero sí para revenue serio.

Tabla

debacu_eval_revenue_price_changes

```
create table if not exists public.debacu_eval_revenue_price_changes (  
  id uuid primary key default gen_random_uuid(),  
  org_id uuid not null references public.debacu_eval_organizations(id) on delete  
  cascade,  
  property_id uuid not null references public.debacu_eval_properties(id) on  
  delete cascade,
```

```

    room_type_id uuid not null references
public.debacu_eval_property_room_types(id) on delete cascade,
    calendar_date date not null,
    old_price numeric(10,2) null,
    new_price numeric(10,2) not null,
    change_reason text null,
    change_source text not null default 'MANUAL',
    changed_by uuid null,
    created_at timestamptz not null default now()
);
create index if not exists
debacu_eval_revenue_price_changes_property_room_date_idx
on public.debacu_eval_revenue_price_changes(property_id, room_type_id,
calendar_date);

```

change_source

Valores útiles:

- MANUAL
 - SEASON_RULE
 - EVENT_RULE
 - BULK_UPDATE
 - DEMAND_ADJUSTMENT
 - CSV_SYNC
-

9) Qué hacer con días ya vendidos

Aquí hay una decisión importante.

No bloquee toda la celda si ya se vendió una parte

Porque si tienes 5 habitaciones del tipo y solo has vendido 2, te quedan 3 por vender.

Entonces necesitas calcular:

- rooms_count del tipo
- units_sold ese día
- $\text{units_available} = \text{rooms_count} - \text{units_sold}$

Regla recomendada

Si $\text{units_available} > 0$

la celda sigue editable

si units_available = 0

la celda queda bloqueada para precio manual

Eso es mucho más real que bloquear toda la celda al primer movimiento.

10) Cómo conectarlo con PriceCalendarPage

Paso 1

Ampliar temporadas y eventos con pricing fields.

Paso 2

Crear un helper TS, por ejemplo:

revenuePricingRules.service.ts

con función tipo:

```
type PricingContextRule = {
  sourceType: "SEASON" | "EVENT";
  sourceId: string;
  name: string;
  priority: number;
  pricingOperation: "INCREASE" | "DECREASE" | "SET" | null;
  pricingAdjustmentType: "PERCENT" | "FIXED" | null;
  pricingAdjustmentValue: number | null;
};

export function applyPricingRule(
  basePrice: number,
  rule: PricingContextRule | null
): number {
  if (!rule || !rule.pricingOperation || !rule.pricingAdjustmentType ||
    rule.pricingAdjustmentValue == null) {
    return basePrice;
  }
  const value = Number(rule.pricingAdjustmentValue);
  if (rule.pricingOperation === "SET") {
    return rule.pricingAdjustmentType === "FIXED" ? value : basePrice;
  }
  if (rule.pricingAdjustmentType === "PERCENT") {
    const delta = (basePrice * value) / 100;
    return rule.pricingOperation === "INCREASE"
      ? basePrice + delta
      : basePrice - delta;
  }
  if (rule.pricingAdjustmentType === "FIXED") {
    return rule.pricingOperation === "INCREASE"
      ? basePrice + value
```

```
        : basePrice - value;
    }
    return basePrice;
}
```

Paso 3

Cuando pintas una celda:

- buscas precio grabado
 - si no hay, usas `base_price`
 - buscas regla activa del día
 - calculas precio sugerido
 - muestras resultado
-

11) Qué tocar en la pantalla de Eventos y Temporadas

Además del color, prioridad e impacto, añade estos campos al formulario:

- Operación
 - Subir
 - Bajar
 - Fijar
- Tipo de ajuste
 - %
 - importe fijo
- Valor
 - número

Ejemplos

- Temporada Alta → Subir + % + 50
- Navidad → Fijar + fijo + 220
- Temporada Baja → Bajar + % + 20

Eso encaja perfecto con la operativa que quieres.

12) Plan de implementación realista

Fase siguiente inmediata

1. SQL para ampliar seasons y events
2. actualizar servicios TS
3. actualizar EventsSeasonsPage con campos de pricing
4. actualizar revenueCalendarContext.service.ts o crear servicio nuevo que traiga también pricing
5. calcular precio por defecto en PriceCalendarPage

Fase después

6. tabla booking_lines
7. importador CSV de ventas
8. cálculo de units_sold por día/tipo
9. bloquear solo cuando no queden unidades disponibles

Fase posterior

10. histórico de cambios de precio
 11. analítica real:
 - precio objetivo
 - precio publicado
 - precio vendido
 - ADR real
 - pickup
 - desviación
-

Mi recomendación directa

El siguiente paso correcto no es tocar todavía CSV.
Es este:

Añadir pricing fields a temporadas y eventos y hacer que el calendario calcule el precio por defecto automáticamente

Eso te deja una operativa ya útil y demostrable.

Después de eso, montas la tabla de ventas reales por CSV.

Si quieres, en el siguiente mensaje te doy el bloque ya ejecutable con:

- SQL exacto para seasons/events
- tipos TS
- cambios en `revenueEvents.service.ts`
- cambios en `revenueSeasons.service.ts`
- y el helper para aplicarlo en `PriceCalendarPage`.

Pero **no tienes datos de ventas reales**.

Sin ventas reales no hay:

- ADR
- Pickup
- Elasticidad
- IA revenue
- Forecast

Diseño correcto de la tabla `revenue_sales`

SQL recomendado:

```
create table if not exists public.debacu_eval_revenue_sales (  
  id uuid primary key default gen_random_uuid(),  
  org_id uuid not null  
    references public.debacu_eval_organizations(id)  
    on delete cascade,  
  property_id uuid not null,  
  room_type_id uuid not null  
    references public.debacu_eval_property_room_types(id),  
  stay_date date not null,  
  reservation_id text null,  
  price_sold numeric not null,  
  channel text null,  
  created_at timestamptz not null default now()  
);
```

Índices necesarios

Para analytics rápidos:

```
create index if not exists idx_revenue_sales_org_date
on public.debacu_eval_revenue_sales (org_id, stay_date);
create index if not exists idx_revenue_sales_room_type
on public.debacu_eval_revenue_sales (room_type_id);
create index if not exists idx_revenue_sales_channel
on public.debacu_eval_revenue_sales (channel);
```

Servicio TypeScript

src/modules/revenue-intelligence/services/revenueSales.service.ts

Ejemplo:

```
export type RevenueSale = {
  id: string
  org_id: string
  property_id: string
  room_type_id: string
  stay_date: string
  reservation_id?: string
  price_sold: number
  channel?: string
}
export async function getRevenueSales(
  orgId: string,
  from: string,
  to: string
) {
  const { data, error } = await supabase
    .from("debacu_eval_revenue_sales")
    .select("*")
    .eq("org_id", orgId)
    .gte("stay_date", from)
    .lte("stay_date", to)
  if (error) throw error
  return data
}
```

CSV PMS que alimentará revenue_sales

Formato recomendado:

```
stay_date,room_type,price_sold,channel
2026-08-01,DCM,135,BOOKING
2026-08-01,DCM,135,WEB
2026-08-01,DCM,120,EXPEDIA
```

Importación:

CSV PMS

↓

```
import_revenue_sales Edge Function
↓
debacu_eval_revenue_sales
```

Con esto podrás calcular

ADR

```
avg(price_sold)
```

Pickup

reservas acumuladas por fecha.

Elasticidad de precio

comparar:

precio vs ocupación

Revenue Leakage (muy potente)

Ejemplo:

precio recomendado = 150€

precio vendido = 110€

→ alerta automática.

Esto es algo que casi ningún RMS pequeño tiene

Debacu puede detectar automáticamente:

"estás vendiendo por debajo del mercado"

o

"estás perdiendo revenue en este canal"

PROMPT CONTINUIDAD PARA MAÑANA

Copia esto exactamente para seguir:

PROMPT CONTINUIDAD — Debacu Revenue Intelligence (Ventas reales)

Seguimos con Debacu Evaluation360.

Estado actual del sistema Revenue:

YA IMPLEMENTADO

[1] Tipos de habitación

tabla `debacu_eval_property_room_types`

[2] Temporadas / eventos

tabla `debacu_eval_property_seasons`

[3] Contexto del calendario

vista `debacu_eval_property_calendar_context_v`

[4] Motor de pricing funcionando

Precio final:

`base_price`

+ `season_adjustment`

UI funcionando:

- punto de color en calendario
- +% temporada
- precio automático

[5] Arquitectura de planes y seats finalizada

Vista:

`debacu_eval_org_entitlements_v`

Devuelve:

`org_id`

`customer_id`

`plan_code`

`max_users`

`seats_used`

`subscription_status`

Esto permite enforcement backend de:

- planes
- seats
- permisos

PRÓXIMO PASO

Implementar **ventas reales del hotel**.

Nueva tabla:

`debacu_eval_revenue_sales`

Campos:

`org_id`

`property_id`

room_type_id
stay_date
reservation_id
price_sold
channel
created_at

Objetivo:

alimentar:

ADR

Pickup

Elasticidad

Revenue leakage AI

Fuente de datos:

CSV exportado desde PMS.

OBJETIVO DEL SIGUIENTE BLOQUE

Diseñar:

- ☐ tabla revenue_sales
 - ☐ servicio revenueSales.service.ts
 - ☐ importador CSV PMS
 - ☐ base para algoritmos de revenue AI.
-

Si quieres, mañana también te explico algo **muy potente que puede hacer Debacu y casi ningún RMS hace bien**:

IA que detecta automáticamente

- precios demasiado bajos
- precios demasiado altos
- canales que destruyen ADR
- fechas donde estás perdiendo dinero.

Eso sería probablemente **la funcionalidad más diferencial de Debacu**.

1. Meter datos de prueba en debacu_eval_revenue_sales

Sin datos no puedes validar nada. Haz primero un seed pequeño pero útil, no 200 filas basura.

Necesitas al menos:

- 2 propiedades

- 2 o 3 room types por propiedad
- 10 a 20 días distintos
- varios canales (WEB, BOOKING, EXPEDIA)
- precios distintos para ver ADR real
- algún día con precio alto y otro con precio bajo

Ejemplo mínimo:

```
insert into public.debacu_eval_revenue_sales
(org_id, property_id, room_type_id, stay_date, reservation_id, price_sold,
channel)
values
('ORG_UUID', 'PROPERTY_UUID', 'ROOMTYPE_UUID', '2026-08-01', 'RES-001', 135, 'WEB'),
('ORG_UUID', 'PROPERTY_UUID', 'ROOMTYPE_UUID', '2026-08-01', 'RES-
002', 120, 'BOOKING'),
('ORG_UUID', 'PROPERTY_UUID', 'ROOMTYPE_UUID', '2026-08-02', 'RES-
003', 145, 'EXPEDIA'),
('ORG_UUID', 'PROPERTY_UUID', 'ROOMTYPE_UUID', '2026-08-03', 'RES-004', 160, 'WEB');
```

2. Crear el primer cálculo real: ADR

No empieces por IA todavía. Primero métricas base.

Qué debe devolver

Por rango de fechas y propiedad:

- total_sales
- total_revenue
- adr

Fórmula

$ADR = \text{sum}(\text{price_sold}) / \text{count}(*)$

Servicio

Haz una función tipo:

getRevenueSalesSummary(propertyId, from, to)
que agrupe todo y devuelva una tarjeta simple.

3. Crear el segundo cálculo: ADR por canal

Esto ya te da valor real muy rápido.

Agrupar por channel y calcular:

- ventas

- revenue
- ADR

Así verás cosas como:

- WEB ADR 148
- BOOKING ADR 126
- EXPEDIA ADR 119

Eso ya empieza a enseñar **fuga de revenue**.

4. Crear una vista o helper para agregados

Si lo haces todo en frontend, luego será un lío. Mejor crear una vista SQL o una consulta centralizada.

Ejemplo de vista simple:

```
create or replace view public.debacu_eval_revenue_sales_daily_v as
select
  org_id,
  property_id,
  room_type_id,
  stay_date,
  channel,
  count(*)::int as sales_count,
  sum(price_sold)::numeric as revenue_total,
  avg(price_sold)::numeric as adr
from public.debacu_eval_revenue_sales
group by
  org_id, property_id, room_type_id, stay_date, channel;
```

No es obligatoria, pero ayuda bastante.

5. Pintarlo en UI cuanto antes

Aunque sea feo al principio.

Pantalla mínima:

- selector de propiedad
- rango de fechas
- 3 cards:
 - ventas
 - revenue
 - ADR
- tabla por canal:
 - canal

- ventas
- revenue
- ADR

Si no lo visualizas pronto, acabarás metiendo backend sin comprobar si realmente sirve.

6. Después sí: Pickup

Una vez tengas ventas reales cargadas, el siguiente paso bueno es:

- reservas por `stay_date`
- evolución por fecha
- ver si el ritmo de ventas acelera o cae

Pero pickup serio necesita una dimensión temporal de captura. Con la tabla actual tienes ventas, pero no el “cuándo se vendió” salvo `created_at`.

Eso significa:

realidad

Con tu tabla actual puedes hacer una versión básica de pickup usando `created_at`.

limitación

No podrás hacer pickup fino tipo RMS profesional si no guardas además:

- `booking_date` o `sold_at`
- idealmente `checkin_date` y `lead_time`

Ahora mismo yo añadiría ya este campo antes de seguir:

```
alter table public.debacu_eval_revenue_sales
add column if not exists sold_at timestamptz null;
```

Y si vas a importar desde PMS, mejor también:

```
alter table public.debacu_eval_revenue_sales
add column if not exists booking_date date null;
```

Porque `created_at` es cuándo insertas en Debacu, no necesariamente cuándo se vendió en el PMS. Esa diferencia importa.

7. Lo más importante: evitar un modelo pobre desde el principio

Tu tabla actual vale para arrancar, pero para revenue de verdad yo no la dejaría solo así.

Campos que merece la pena añadir ya, antes de que se llene:

```
alter table public.debacu_eval_revenue_sales
add column if not exists booking_date date null,
add column if not exists sold_at timestamptz null,
```

```
add column if not exists nights integer null,  
add column if not exists guests integer null,  
add column if not exists currency text null default 'EUR';  
Y si quieres algo más sólido:  
add column if not exists rate_plan text null,  
add column if not exists is_refundable boolean null;
```

Por qué

Porque luego querrás analizar:

- ADR por estancia
- segmento
- antelación
- régimen
- canal
- tarifa

Si no guardas eso, luego tocará rehacer imports.

Mi recomendación directa

El siguiente bloque debería ser este:

1. seed de datos reales de prueba
2. cards KPI: ventas, revenue, ADR
3. tabla ADR por canal
4. añadir `booking_date` o `sold_at` antes de seguir con pickup

Eso es sensato. Ponerse ya con IA sin eso sería humo.

Prompt de continuidad

PROMPT CONTINUIDAD — Debacu Revenue Intelligence (Revenue Sales ya creada)

Seguimos con Debacu Evaluation360 – módulo Revenue Intelligence.

Estado actual:

YA IMPLEMENTADO

- 1] tabla `debacu_eval_revenue_sales` creada
- 2] índices creados
- 3] servicio `revenueSales.service.ts` creado

Próximo objetivo inmediato:

conectar `revenue_sales` con métricas reales visibles en UI.

Orden del siguiente bloque:

1. cargar datos de prueba en `debacu_eval_revenue_sales`
2. construir resumen KPI:
 - `total_sales`
 - `total_revenue`
 - `ADR`
3. construir tabla ADR por canal
4. preparar base para pickup

Importante:

antes de pickup serio revisar si la tabla necesita estos campos adicionales:

- `booking_date`
- `sold_at`
- `nights`
- `guests`
- `currency`

porque `created_at` no siempre representa la fecha real de venta del PMS.

Objetivo del siguiente paso:

crear pantalla o bloque UI que muestre:

- cards de ventas / revenue / ADR
- tabla por canal
- filtro por propiedad y rango de fechas

Después de eso, siguiente fase:

Pickup + Revenue Leakage básico.

Si quieres, en el siguiente mensaje te doy directamente el SQL de seed y la estructura exacta del componente React para la pantalla KPI.

El enfoque que propones es **correcto y sensato**: avanzar **pantalla por pantalla, conectando cada una a datos reales antes de seguir**. Esa disciplina evita que el proyecto se vuelva caótico. Te propongo un **orden claro de ejecución** para que mañana continúes sin perder el hilo.

1. Orden de desarrollo del módulo Revenue

Mantén esta secuencia estricta.

Paso 1 (mañana)

Canales & Segmentos

Objetivo: conectar la primera pantalla analítica real.

Datos que ya tienes en `debacu_eval_revenue_sales`:

- `channel`
- `price_sold`
- `stay_date`
- `property_id`

Esto permite calcular:

- `total_sales`
- `total_revenue`
- `ADR`
- distribución por canal

Consulta base (ejemplo):

```
select
  rs.channel,
  count(*) as total_sales,
  sum(rs.price_sold) as total_revenue,
  round(avg(rs.price_sold),2) as adr
from debacu_eval_revenue_sales rs
where rs.property_id = :property_id
group by rs.channel
order by total_revenue desc;
```

Esto alimenta:

- tarjetas KPI
- gráfico barras canal
- tabla resumen

No necesitas nada más todavía.

Paso 2

Mensual

Segunda pantalla.

Datos necesarios:

- `month`
- `revenue`
- `ADR`

- RN (rooms sold)

Consulta base:

```
select
    date_trunc('month', stay_date) as month,
    count(*) as rn,
    sum(price_sold) as revenue,
    round(avg(price_sold),2) as adr
from debacu_eval_revenue_sales
where property_id = :property_id
group by month
order by month;
```

Esto alimenta:

- tabla mensual
- gráfico evolución revenue

Lo que **no tienes aún**:

- OCC
- RevPAR
- YoY

Eso se añade más adelante.

Paso 3

Dashboard Revenue

Solo después de esas dos.

KPIs:

- Revenue total
 - ADR
 - RN
 - canal principal
 - tendencia
-

Paso 4

Día x Día

Aquí ya necesitarás:

- inventario habitaciones
- rooms_total
- occupancy real

Este vendrá después.

Paso 5

Pickup

Necesitará:

booking_date

stay_date

Ahora mismo aún no lo tienes completo.

2. Regla que debes seguir siempre

Nunca empieces una pantalla nueva si la anterior:

- no funciona
- no está conectada
- no está limpia

Siempre:

UI

↓

Query

↓

Service

↓

Hook

↓

Componente

↓

Test

Esto es exactamente lo que estás haciendo y es **la forma profesional de construir SaaS complejos**.

3. Sobre la web pública que estás cambiando

Te doy feedback directo porque aquí hay algo importante.

Tu web actual comunica:

Evaluación

Incidencias

Auditoría

Control

Tu producto real está evolucionando hacia:

AI para hospitality
Revenue intelligence
Risk analytics
Operational signals
Esto **no es lo mismo**.

Para subvenciones e inversores es **mucho mejor la segunda narrativa**.

4. La estructura que muestras (la nueva) es mejor

La nueva landing comunica:

Hero

IA para hoteles

4 pilares

- 1 Evaluación de riesgo operativo
- 2 Revenue intelligence
- 3 Análisis de comportamiento
- 4 Inteligencia artificial aplicada

Esto es **mucho más escalable para inversión**.

Porque posiciona Debacu como:

Hospitality Intelligence Platform
no como:

herramienta de incidencias
que es un mercado pequeño.

5. Ajuste que te recomiendo para subvenciones

Tu mensaje debería quedar así:

Posicionamiento

Debacu

AI-powered operational intelligence for hotels.

Plataforma

4 módulos:

- 1 Risk Intelligence
- 2 Revenue Intelligence

3 Behavior Analytics

4 Decision Engine

Esto es perfecto para:

- CDTI
 - ENISA
 - inversores
 - ayudas digitalización
-

6. Algo muy importante que ya estás haciendo bien

Tienes **tres capas claras**:

1 Web pública

Marketing

Narrativa

Captación

2 App SaaS

Operación real

- pricing
 - analytics
 - ingestión
-

3 Motor AI

Modelos

- anomalías
- revenue
- patrones

Esto es exactamente la arquitectura que se espera en un SaaS AI moderno.

7. Plan claro para mañana

Objetivo único

Implementar:

Canales & Segmentos (real)

Orden:

- 1 query SQL final
- 2 endpoint service
- 3 hook react
- 4 conectar a UI
- 5 validar DEMOHOTEL

Cuando eso funcione:

- ➔ commit
 - ➔ seguir con **Mensual**
-

8. Mi recomendación clave ahora

No intentes avanzar **web pública + SaaS + IA** al mismo tiempo cada día.

Divide el foco.

Ejemplo:

Lunes → producto SaaS

Martes → web marketing

Miércoles → motor AI

Si no, el contexto mental se rompe.

9. Resumen

Tu plan es correcto:

- ✓ avanzar pantalla a pantalla
- ✓ conectar datos reales
- ✓ no saltar pasos
- ✓ mejorar narrativa web para subvenciones

El siguiente paso lógico es:

Revenue → Canales & Segmentos → datos reales → luego Mensual

Si quieres, mañana también puedo ayudarte a diseñar **la query completa que alimentará toda la pantalla Canales & Segmentos** (tarjetas + gráfico + tabla) para que no tengas que rehacer nada después.

